Regis University CPS
CS361 Programming Assignment 5
Due by midnight Sunday of week 5

Between these two programs, where it makes sense, you must demonstrate at least one example each of a:

- **for** loop
- **do-while** loop
- **while** loop

NOTES:

1) Remember that use of global variables is **NOT** allowed.

2) You are also **NOT** allowed to use either the **break** statement or the **return** statement to exit from a loop. All loops must exit ***ONLY*** via their **test condition**.

Program #1

The **natural logarithm of 2** is approximately:

ln(2) = 0.693147180559945309417232121458…

Various formulas exist for calculating the **natural logarithm of 2**. One approach is to calculate the value via a ***sum of fractions***, using the following series. The more terms used in the series, the better the precision of the natural logarithm of 2 that is calculated.

ln(2) = 1/1 – 1/2 + 1/3 – 1/4 + 1/5 – 1/6 + …

As you might have noticed, calculating the value of this series only requires performing elementary arithmetic. Notice that the terms in the denominator consist of increasing integers and each subsequent term alternates between being added to or subtracted from the previously summed terms. For example, the value of natural logarithm of 2, using the first four terms in this series is:

ln(2) = 1/1 – 1/2 + 1/3 – 1/4 = 0.58 *(not very accurate with only 4 terms!)*

The more terms that are used, the closer the approximation will become to natural logarithm of 2's ***actual value*****.**

Requirements:

Write a program based on the previous series that calculates the value of the natural logarithm of 2, given a specified number of terms.

- Tell the user what the program does before prompting for input.

©2014, Regis University, All Rights Reserved
Unauthorized distribution (including uploading to any non-Regis Internet website) violates copyright law

- Use **double** precision floating point values to store all floating point values.

- All user input for this program will be need to be positive, non-zero values. So you will write a ***user-defined*** function to read positive, non-zero input from the user. The user-defined function requirements are:

    - The function will have one input parameter, the prompt for user input. This value will be stored in a **string** type variable (examples: "Enter the number of terms to compute").

        - The prompts can either be stored as constant strings, or may be hardcoded strings in the function calls.

    - The function will prompt the user for the input and then use a loop to verify that the value input is ***positive and non-zero***. If it is not, the function will issue an error message and then re-prompt the user to enter the number again, until a positive, non-zero value is entered.

    - Once valid input has been entered, the value that the user entered will be returned to the **main** function via the return statement.

    - NOTE: All prompting, reading, and validation must be done from within the function.

- Call the above user-defined function to read in the ***number of terms*** in the series to be used for calculating the value of the natural logarithm of 2 (note, the above two formula examples have 6 and 3 terms respectively). Pass in a prompt for terms and return the validated number of terms value to **main**.

- After a valid number of terms has been entered, call the ***same user-defined function*** a second time, to read in a ***display step count*** representing how often the program should display the current total value of the series. For example, if the user inputs 1, the result after ***every*** step (every term) will be output. If the user inputs 10, then the program should output after the 10th, 20th, 30th, etc term calculations (see sample outputs below). Using the same user-defined function will work, because the user input should again be ***positive and non-zero***. Pass in a prompt for the steps and return the validated step value to **main**.

- Output the total value of the series at each step, specified by the display count, displayed to 10 decimal places.

- Output the final value of the natural logarithm of 2 for the sum after the last term in the series, to 10 decimal places.

©2014, Regis University, All Rights Reserved
Unauthorized distribution (including uploading to any non-Regis Internet website) violates copyright law

Input/Output Sample 1:

```
Program will approximate the natural logarithm of 2

Enter the number of terms to compute: 5

Display value after every how many steps? 1


RESULTS at:
    1 steps:  ln(2) = 1.0000000000
    2 steps:  ln(2) = 0.5000000000
    3 steps:  ln(2) = 0.8333333333
    4 steps:  ln(2) = 0.5833333333
    5 steps:  ln(2) = 0.7833333333

Finally, after
    5 steps:  ln(2) = 0.7833333333
```

Input/Output Sample 2:

```
Program will approximate the natural logarithm of 2

Enter the number of terms to compute: 500

Display value after every how many steps? 0
ERROR - Input must be positive and non-zero!  Please try again.

Display value after every how many steps? 150


RESULTS at:
  150 steps:  ln(2) = 0.6898249581
  300 steps:  ln(2) = 0.6914832917
  450 steps:  ln(2) = 0.6920373040

Finally, after
  500 steps:  ln(2) = 0.6921481806
```

(see next page for program 2)

©2014, Regis University, All Rights Reserved
Unauthorized distribution (including uploading to any non-Regis Internet website) violates copyright law

Program #2

The following formula will compute the Total Amount that will be accumulated for a given Monthly Amount saved:

$$\textbf{total} = \frac{(\textbf{factor} - 1)\ \textbf{monthlyAmount}}{\textbf{monthlyInterestRate}}$$

Notice that his formula also requires that you first calculate a **factor** and then use it in your formula. The **factor** formula is:

factor = exp(numberOfMonths * log(1 + monthlyInterestRate))

exp and **log** are predefined mathematical functions, located in the C++ <**cmath**> library.

For hand calculating results to test your program, the equivalent algebraic equation is:

$$\textbf{factor} = (1 + \textbf{monthlyInterestRate})^{\textbf{numberOfMonths}}$$

Write a program to output accumulated values for each month, given a set amount saved each month, until the accumulated amount ***reaches*** a set goal amount.

Discussion about comparing floating point values:

Given two floating point values:

```
num = 9.99999999
num2 = 10.00000000
```

If both are displayed to two decimal places, you will see:

`10.00` and `10.00`

However, if you ***compare*** the same two values within your code, to see if they are equal, the comparison test will be **false** (i.e. they are not equal).

This can cause problems for programs that compare floating point values.
But we can fix this problem by forcing the ***stored value*** to be rounded to 2 decimal places (not just the output value).

The formula:

```
num = floor((num * 100) + 0.5) / 100.0;
```

will change the value stored in **num** to be:

```
num = 10.00000000
```

And then if you compare **num** with **num2**, they will be found to be equal.

©2014, Regis University, All Rights Reserved
Unauthorized distribution (including uploading to any non-Regis Internet website) violates copyright law

Requirements:

Ensure that you define **constants** for all fixed values. After defining the constants:

- Use only the constant ***name*** in your code (no hardcoded constant ***values***).
- The usage of only constant ***names*** includes references in the executable code, the prompts and error messages, and the comments -- no hardcoded values anywhere!

Use the **double** data type for all floating point values.

The program should:

- Tell the user what the program does before prompting for input.
- Read from the user the amount that will be saved each month.
    - The user must enter a value between $5 and $5000 inclusive. If they do not, issue an error message and then re-prompt the user to enter the number again, until a valid value is entered.
- Read from the user the annual interest rate.
    - The user must enter a value between 1% and 25% inclusive. If they do not, issue an error message and then re-prompt the user to enter the number again, until a valid value is entered.
    - After reading a valid value, the program will need to convert the annual interest rate to a monthly interest rate (divide by 100 to get a fractional interest rate, and then divide that by 12 for monthly interest) before using it in the formulas.
- Read from the user the goal amount they wish to reach.
    - The user must enter a positive amount of no more than $ 999,999.99 (i.e. under 1 million). If they do not, issue an error message and then re-prompt the user to enter the number again, until a valid value is entered.
- Determine the **factor** and the **total amount accumulated** as **calls** to two ***different*** user-defined **functions**.
    - The first function will return the **factor** for a specific number of months.
    - The second function will return the **total amount accumulated** for a specific number of months
    - Pass any required information, including the number of months to calculate the value for, to the functions as parameters.
- Display the **month number** and the **total amount accumulated**, for every month until the goal is reached, formatted as shown in the output example below.
    - NOTE: Use the rounding formula provided above to round the value returned from the function and stored in the **total amount accumulated** variable to 2 decimal places, before comparing it to any other value.

©2014, Regis University, All Rights Reserved
Unauthorized distribution (including uploading to any non-Regis Internet website) violates copyright law

- The program should cease displaying these values as soon as the **total amount accumulated** is ***at or above*** the **goal**.
- After the **goal** has been reached, display the annual interest rate, savings per month, total number of months, and final amount accumulated formatted as shown in the output example below.

Sample Input

```
Program will calculate the amount accumulated every month you save,
until you reach your goal.


Enter the dollar amount to be saved each month: 1
Invalid amount -- must save between 5 and 5000 dollars a month.  Try again.

Enter the dollar amount to be saved each month: 333.33

Enter the annual interest rate (between 1 and 25): 5.55

Enter the dollar amount of your goal: 5000
```

Sample Output

```
Month    Total
Number   Accumulated
------   -----------
   1         333.33
   2         668.20
   3        1004.62
   4        1342.60
   5        1682.14
   6        2023.25
   7        2365.94
   8        2710.21
   9        3056.07
  10        3403.54
  11        3752.61
  12        4103.29
  13        4455.60
  14        4809.54
  15        5165.11

At an annual interest rate of        5.550 %
With a monthly savings of          333.33
It will take                          15 months
To reach your goal of              5000.00
With a final accumulation of       5165.11
```

Note that dollar amounts are calculated to the nearest penny (i.e. rounded to 2 decimal places), and the annual interest rate is displayed to 3 decimal places with a % after it.

©2014, Regis University, All Rights Reserved
Unauthorized distribution (including uploading to any non-Regis Internet website) violates copyright law

Program and Function Documentation (both programs)

1) Include **top of program** comments as specified in course Content section 1.8.

2) Comment any constants/variables whose names are not completely descriptive.

3) You must also include a comment above **each user-defined function** (above the function **definition**, ***not*** above the function **prototype**) that includes:

- Dividing lines
- A description of what the function does
- The name and a description of each ***parameter***
- The name (if there is one), and a description of what is returned from the function via the **return** statement (if anything).

Example:

```
// ------------------------------------------------------
// Description: SumTwo will sum two numbers
// Input parameters: num1, num2 - numbers to add
// Returns:  sum - sum of num1 and num2
// ------------------------------------------------------
int SumTwo (int num1, int num2)
```

Testing

As part of your submission for this week, you must write a **test plan** for this program (program 2). Submit your test plan as an MS Word document or an MS Excel spreadsheet.

Your test plan must include:

- the ***rationale*** for testing each condition within the program
- a list of the specific test cases

Each test case must include:

- the exact ***user input used*** for the test
- the exact ***output expected***

A test plan for last week's program 2 is included in the online Content as a sample (see section 5.7.3).

Submission

This programming assignment is due by midnight Sunday. Submit your program source code to the ***Prog Assn 5*** folder in the ***Dropbox*** area of the online course. **All** files should be attached to **one** submission for programming assignment 5.

©2014, Regis University, All Rights Reserved
Unauthorized distribution (including uploading to any non-Regis Internet website) violates copyright law

Before submitting your program files, you MUST name them as follows:

Lastname-wk5-prog1.cpp
Lastname-wk5-prog2.cpp

And your test plan file name MUST be:

Lastname-wk5-p2testplan.xxx

For example:

Smith-wk5-prog1.cpp
Smith-wk5-prog2.cpp
Smith-wk5-p2testplan.doc

Grading

The rubric that will be used to grade your program is included on the next pages.

©2014, Regis University, All Rights Reserved
Unauthorized distribution (including uploading to any non-Regis Internet website) violates copyright law

CS 361 Control Structures
Programming Assignment 5 Rubric

Program 1: (42 pts)

Rating Rating Category	Exemplary	Proficient	Basic (needs work)	Not Demonstrated
Data Storage Points	Variables defined locally, using appropriate data types. Followed all coding standards naming conventions.	A few minor errors in data typing, naming, or assignment.	One or more major errors, OR majority of names did not follow naming conventions.	Identifier names are not descriptive, and/or there is a multitude of errors OR defined global variable(s).
User Input: Definition of User Input Validation Function	One generic function defined as specified by the requirements, with proper parameter passing. Loop statement prompts using input argument and validates user input correctly. Valid value returned.	Minor errors in function definition (parameter or return value), or in looping statements, or function name not descriptive.	Defined a separate function for each required input, and did not pass the prompt as an input parameter OR other major errors.	Required function was not implemented at all. Input was simply read from within the main function.
User Input: Calling of User Input Validation Function	Generic function called multiple times, with different prompts as input arguments. Correctly captured returned values.	Minor error in function calls or capture of returned values.	Function was not called. Task was simply performed from within the main function, but validation was still performed.	Function was not called. Task was simply performed from within the main function, and NO input validation was performed.
Program Logic: if and loop statements, and calculations	Logic accomplishes all program requirements. All math computations compute correct results. All if and loop statements are written correctly.	One or two minor errors in the program logic, including incorrect math computations, incorrect if statement logic, and/or incorrect loop logic.	Multiple errors in the program logic, including incorrect math computations, incorrect if statement logic, and/or incorrect loop logic.	Logic accomplishes few or none of the program requirements.
Program Output	Displays all of the required output, formatted according to program requirements.	One or two minor errors in the output display.	Multiple errors in the output display.	Displays completely incorrect output, or does not display any output at all.
C++ Constructs / Readability / Miscellaneous	Code is organized and easy to follow. There is no unnecessary code and all code is spaced and indented correctly.	Code is fairly easy to read, but there are some spacing, indentation, and/or other minor issues, or extraneous code.	Code is somewhat difficult to read or has some other major issue.	The code is formatted so badly that it is extremely hard to read or has multiple major other issues.

©2014, Regis University, All Rights Reserved
Unauthorized distribution (including uploading to any non-Regis Internet website) violates copyright law

Program 2: (52 pts)				
Data Storage Points	Constants defined for fixed values. Variables defined locally, using appropriate data types. Followed coding standards naming conventions.	A few minor errors in data typing, naming, or assignment OR missing one or two necessary constants for fixed value.	One or more major errors, OR majority of names did not follow naming conventions, OR missing multiple constants for fixed values.	Constants not used, identifier names are not descriptive, and there is a multitude of errors OR defined global variable(s).
Program Input	Displays descriptive prompt. Validates all user input, reprompting as necessary until good values are entered. Uses constants, not hardcoded values, in prompts, error messages, and test conditions.	Reads input, but does not validate one of the values correctly, OR hardcodes values in prompt, error message, or test condition instead of using defined constants.	Reads input, but does not validate any of the values correctly OR hardcodes values in ALL prompts, error messages, and test conditions.	Hardcoded values used instead of reading input, OR reading of input fails completely.
Function Implementation: User-defined functions	Both functions defined and called as specified by the requirements, with proper parameter passing. Correct saving of values returned to main.	One or two minor errors in function definitions, function calls, or saving values returned.	Many errors in function definitions, function calls, or saving values returned OR only implemented ONE of the required functions.	None of the required functions were implemented. Tasks were simply performed from within the main function.
Program Logic: if and loop statements, and calculations	The program logic accomplishes all program requirements. All math computations compute correct results. Correctly written if and loop statements, which exit correctly via the test conditions.	One or two minor errors in the program logic, including incorrect math computations, incorrect if statement logic, and/or incorrect loop logic.	Many errors in the program logic, including incorrect math computations, incorrect if statement logic, and/or incorrect loop logic.	Logic accomplishes few or none of the program requirements.
Output	Displays all of the required output, formatted according to program requirements.	One or two minor errors in the output display.	Multiple errors in the output display.	Displays completely incorrect output, or does not display any output at all.
C++ Constructs / Readability / Miscellaneous	Code is organized and easy to follow. There is no unnecessary code and all code is spaced and indented correctly.	Code is fairly easy to read, but there are some spacing, indentation, and/or other minor issues, or extraneous code.	Code is somewhat difficult to read or has some other major issue.	The code is formatted so badly that it is extremely hard to read or has multiple major other issues.
Test Plan	Test plan includes rationale and tests that validate ALL logic.	Missing parts of rationale or missing tests of some logic.	Missing all rationale and/or missing many tests of the logic.	No test plan submitted.

©2014, Regis University, All Rights Reserved
Unauthorized distribution (including uploading to any non-Regis Internet website) violates copyright law

Both Programs (6 pts)				
Loop Usage	Between the two programs, implemented at least one each of: for, while, and do-while loops	Only correctly implemented 2 of the 3 loop types.	Only correctly implemented one loop type.	No loops implemented correctly.

Documentation	Top of program comments include all required components. Top of function comments explain what function will do, and describe all parameters and return values. Inline comments are included as necessary.	Program description missing some details about the input, output, or processing OR top of function comments vague or missing some details about parameters and return values OR missing other minor necessary comment.	Documentation has many problems with missing, incomplete and/or incorrect items in program description, top of function comments, and/or inline comments.	Only a few (or no) comments in program.

NOTE: Deductions for late submissions will be applied (-2% per day late)
But programs will **not be accepted** if they are submitted **more than 5 days** late (grade will be 0).

©2014, Regis University, All Rights Reserved
Unauthorized distribution (including uploading to any non-Regis Internet website) violates copyright law